

Bàn sâu về quy hoạch động

Li Gang

Nguồn gốc: An in-depth discussion of dynamic programming

IOI China candidate team papers / 2000 / Li Gang.pdf

Từ khóa. Quy hoạch động; trạng thái.

Tóm Tắt

Bài viết này thảo luận một kỹ thuật giải bài rất hiệu quả: quy hoạch động. Hiệu suất giải bài cao của kỹ thuật này luôn được quan tâm nhiều. Trước hết, bài viết bàn về nền tảng lý thuyết của quy hoạch động, và nêu hai tiền đề để một bài toán có thể giải bằng quy hoạch động: cấu trúc con tối ưu và tính không hậu hiệu. Tiếp đó, bài viết xét hai vấn đề thường gặp trong ứng dụng thực tế: chọn trạng thái và lưu trữ trạng thái trong quy hoạch động. Từ các thảo luận ấy, bài viết dẫn đến phương pháp tư duy cơ bản của quy hoạch động: “không làm lại việc đã làm”, cũng như nguyên nhân khiến kỹ thuật quy hoạch động có tốc độ đáng kinh ngạc khi giải bài: “xử lý phần dư thừa trong việc xét các khả năng, đạt tới giới hạn về tốc độ”. Cuối cùng, bài viết trình bày các bước tổng quát để giải bài quy hoạch động: suy nghĩ, lập kế hoạch, và thực hiện.

Contents

1	Dẫn nhập	1
2	Nền tảng lý thuyết của quy hoạch động	3
2.1	Cấu trúc con tối ưu	4
2.2	Tính không hậu hiệu	4
3	Ứng dụng thực tế của quy hoạch động	5
3.1	Chọn trạng thái	5
3.2	Lưu trữ trạng thái	6
4	Suy nghĩ sâu hơn về quy hoạch động	8
5	Kết luận	10
A	Phụ lục	10
A.1	Ví dụ 1	10
A.2	Ví dụ 2	14

A.3 Ví dụ 3	15
A.4 Ví dụ 4	20
A.5 Ví dụ 5	29
A.6 Ví dụ 6	32

1 Dẫn nhập

Trong các kỳ thi tin học, đặc biệt trong vài năm gần đây, quy hoạch động thường xuyên được nhắc đến như một công cụ giải bài. Phạm vi ứng dụng của nó ngày càng rộng, mức độ ứng dụng cũng ngày càng sâu. Vậy quy hoạch động khác những thuật toán khác ở điểm nào? Giá trị ứng dụng cụ thể của nó là gì? Bài viết này sẽ thảo luận các câu hỏi đó.

Trước hết, ta nhận biết quy hoạch động qua một bài toán cụ thể.

Ví dụ 1. Hình 1 cho một bản đồ. Mỗi đỉnh trên bản đồ biểu diễn một thành phố, đường nối giữa hai thành phố biểu diễn đường đi, và số trên đường nối biểu diễn chiều dài đường. Nay ta muốn đi từ thành phố A đến thành phố E. Đi thế nào để quãng đường ngắn nhất, và độ dài ngắn nhất là bao nhiêu?

Giả sử:

$Dis[X]$ là độ dài đường ngắn nhất từ thành phố X đến E,

$Map[I, J]$ là khoảng cách giữa hai thành phố I, J.

Nếu $Map[I, J] = 0$ thì hai thành phố không thông nhau.

Bài toán này có thể làm bằng tìm kiếm; chương trình rất dễ viết:

Var

Se: tap thanh pho chua tham;

Function Long(Who: thanh pho hien tai): Integer;

Begin

If Who=E Then Search:=0

Else

Begin

Min:=Maxint;

For I qua moi thanh pho Do

If (Map[Who,I]>0) And (I In Se) Then

Begin

Se:=Se-[I];

J:=Map[Who,I]+Long(I);

Se:=Se+[I];

If J<Min Then Min:=J;

End;

Long:=Min;

End;

End;

Begin

Se:=tap moi thanh pho tru A;

Dis[A]:=Long(A);

End.

Hiệu suất của chương trình này ra sao? Ta thấy mỗi lần, trừ các thành phố đã thăm, mọi thành phố khác đều phải được xét, nên độ phức tạp thời gian là $O(N!)$. Đây là một thuật toán cấp mũ. Vậy có thuật toán tốt hơn không?

Trước tiên hãy quan sát thuật toán này. Khi tìm đường ngắn nhất từ B_1 đến E , ta đã tìm đường ngắn nhất từ C_2 đến E ; khi tìm đường ngắn nhất từ B_2 đến E , ta lại tìm một lần nữa đường ngắn nhất từ C_2 đến E . Nghĩa là đường ngắn nhất từ C_2 đến E bị tính hai lần. Tương tự, trong quá trình tìm đường ngắn nhất từ C_1, C_2 đến E , đường ngắn nhất từ D_1 đến E cũng bị tính hai lần. Trong toàn bộ chương trình, đường ngắn nhất từ D_1 đến E bị tính bốn lần. Lãng phí lớn biết bao! Nếu trong quá trình giải, ta đồng thời ghi lại độ dài đường ngắn nhất đã tìm được và gọi lại bất cứ lúc nào thì sẽ tiện lợi hơn nhiều.

Từ đó sinh ra một ý tưởng mới: lần lượt suy ra từng giá trị Dis từ sau ra trước, cho đến khi suy ra Dis[A]. Ý tưởng này quả thật tốt, nhưng khoan đã: thế nào là “từ sau ra trước”?

“Sau” và “trước” là thứ tự do ta tự gán cho các thành phố. Khi thứ tự của hai thành phố I, J được định là I “trước”, J “sau”, thì phải thỏa điều kiện:

$$I, J \text{ không thông nhau, hoặc } \text{Dis}[I] + \text{Map}[I, J] \geq \text{Dis}[J].$$

Nếu I, J thông nhau và $\text{Dis}[I] + \text{Map}[I, J] < \text{Dis}[J]$, điều đó cho thấy Dis[J] còn có phương án tốt hơn; nhưng J lại nằm sau I , nên phương án ấy không thể được suy ra, làm ảnh hưởng đến lời giải cuối cùng. Vậy ta chia thứ tự trước sau như thế nào?

Như Hình 2, các thành phố được chia thành nhiều giai đoạn. Ta có thể dùng giai đoạn để biểu diễn thứ tự của mỗi thành phố, vì cách chia giai đoạn có hai tính chất sau:

1. Giá trị ở giai đoạn I chỉ liên quan đến giai đoạn $I + 1$, và giá trị ở giai đoạn I chỉ gây ảnh hưởng đến giá trị ở giai đoạn $I - 1$.
2. Thứ tự giữa các giai đoạn là xác định; không thể đổi chỗ tùy ý hai giai đoạn bất kỳ.

Từ hai tính chất này có thể suy ra rằng dùng giai đoạn làm thứ tự “trước” và “sau” thỏa điều kiện vừa nêu. Vì vậy, ta dùng giai đoạn làm thứ tự của mỗi thành phố, rồi truy ngược từ giai đoạn 3 đến giai đoạn 1, sau đó suy ra Dis[A].

Công thức là

$$\text{Dis}[X] = \min\{\text{Dis}[Y] + \text{Map}[X, Y] : Y \text{ là thành phố ở giai đoạn kế tiếp và thông với } X\}.$$

Có thể xem E là giai đoạn 4 và A là giai đoạn 0.

Chương trình:

```
Dis[E]=0
For X= moi thanh pho tu giai doan 3 xuong giai doan 0 Do
Begin
  Dis[X]:=Maxint;
  For Y= moi thanh pho o giai doan ke tiep cua X Do
    If Dis[Y]+Map[X,Y]<Dis[X] Then
      Dis[X]:=Dis[Y]+Map[X,Y];
End.
```

Độ phức tạp thời gian của chương trình này là $O(N^2)$, nhỏ hơn rất nhiều so với $O(N!)$ của chương trình trước. Thuật toán thứ hai chính là thuật toán quy hoạch động.

2 Nền tảng lý thuyết của quy hoạch động

Qua ví dụ trên, ta đã có nhận thức ban đầu về quy hoạch động: nó xử lý một “bài toán quyết định nhiều giai đoạn”. Bây giờ ta định nghĩa cụ thể một số khái niệm.

Trạng thái (state): biểu diễn tính chất của sự vật, là đại lượng mô tả “đơn vị” trong quy hoạch động. Chẳng hạn trong Ví dụ 1, các thành phố A, B_1, D_2, E đều là những trạng thái riêng lẻ.

Giai đoạn (stage): một tập các trạng thái có tính chất gần nhau và có thể xử lý đồng thời. Thông thường, một bài toán có thể được chia thành vài giai đoạn theo thứ tự xử lý. Ví dụ 1 gồm ba giai đoạn, trong đó giai đoạn 1 chứa hai trạng thái B_1, B_2 . Thực ra giai đoạn chỉ đánh dấu những trạng thái có cùng cách xử lý và không phụ thuộc vào thứ tự xử lý lẫn nhau. Một giai đoạn có thể chứa nhiều trạng thái, cũng có thể chỉ chứa một trạng thái; quan hệ giữa chúng khá giống quan hệ giữa phân tử và nguyên tử.

Phương trình chuyển trạng thái: quy luật biến đổi để trạng thái của giai đoạn trước chuyển thành trạng thái của giai đoạn sau. Đây là phương trình liên quan đến trạng thái của hai giai đoạn kề nhau, và là trung tâm của quy hoạch động.

Quyết định (decision): một hành động lựa chọn nào đó được đưa ra ở mỗi giai đoạn. Đó chính là lựa chọn mà chương trình của ta cần thực hiện.

Bài toán do quy hoạch động xử lý là một bài toán quyết định nhiều giai đoạn: thường bắt đầu từ trạng thái đầu, thông qua việc chọn quyết định ở các giai đoạn giữa, rồi đi đến trạng thái kết thúc. Các quyết định ấy tạo thành một dãy quyết định, đồng thời xác định một tuyến hoạt động để hoàn thành toàn bộ quá trình, thường là tuyến hoạt động tối ưu. Hình 3 minh họa quan hệ này.

Nhìn tổng thể, một thuật toán quy hoạch động nên chứa các quan hệ và thuộc tính cơ bản vừa nêu.

Vậy bài toán quyết định như thế nào mới có thể chia giai đoạn và giải bằng quy hoạch động? Ta nói rằng chỉ khi một bài toán quyết định có các tính chất sau thì mới nên cân nhắc chia giai đoạn và dùng quy hoạch động.

2.1 Cấu trúc con tối ưu

Nghĩa là lời giải tối ưu của một bài toán chỉ phụ thuộc vào lời giải tối ưu của các bài toán con; các lời giải không tối ưu không ảnh hưởng đến việc giải bài toán. Trong Ví dụ 1, nếu muốn tìm giá trị nhỏ nhất của $\text{Dis}[B_1]$, ta chỉ cần biết giá trị nhỏ nhất của C_1, C_2, C_3 , không cần xét các đường khác, vì các đường không tối ưu chắc chắn không ảnh hưởng đến kết quả của $\text{Dis}[B_1]$. Xét thêm một bài toán:

Ví dụ 2. Có bốn điểm A, B, C, D , như Hình 4. Giữa hai điểm kề nhau có hai đường đi C_{2k}, C_{2k-1} ($1 \leq k \leq 3$). Số phía trên đường nối biểu diễn độ dài đường. Ta định nghĩa trong mọi đường từ A đến D , đường có số dư nhỏ nhất khi lấy độ dài chia cho 4 là đường tối ưu. Hãy tìm một đường tối ưu.

Với bài này, nếu vẫn giải theo cách vừa dùng thì sẽ sai. Chẳng hạn, theo tư duy của Ví dụ 1, giá trị tối ưu của A có thể xác định từ giá trị tối ưu của B ; giá trị tối ưu của B là 0, nên giá trị tối ưu của A sẽ là 2. Nhưng thực tế, đường $C_1 - C_3 - C_5$ cho giá trị tối ưu là 0. Vì vậy, đường tối ưu của B không phải đường con của đường tối ưu của A . Nói cách khác, giá trị tối ưu của A không do giá trị tối ưu của B quyết định, nên bài toán không có cấu trúc con tối ưu.

Từ đó có thể thấy không phải mọi bài toán quyết định đều có thể giải bằng quy hoạch động. Chỉ khi một bài toán thể hiện cấu trúc con tối ưu thì quy hoạch động mới có thể là một phương pháp thích hợp.

2.2 Tính không hậu hiệu

Sau khi một bài toán được chia giai đoạn, trạng thái trong giai đoạn I chỉ có thể thu được từ trạng thái trong giai đoạn $I + 1$ thông qua phương trình chuyển trạng thái, không liên quan đến các trạng thái khác, đặc biệt không liên quan đến các trạng thái chưa xảy ra. Đây là tính không hậu hiệu. Thực ra, nếu ta định nghĩa trạng thái của bài toán là các đỉnh trong đồ thị, chuyển giữa hai trạng thái là cạnh, và lượng tăng của trọng số trong quá trình chuyển là trọng số cạnh, thì bài toán thực chất là tìm đường giữa hai đỉnh trong một đồ thị có hướng, không chu trình, có trọng số. Vì có tính không hậu hiệu nên không có chu trình; nếu có chu trình thì dù chia giai đoạn thế nào cũng có thể xuất hiện hậu hiệu. Nói cách khác, đồ thị ấy có thể sắp thứ tự tô pô; ít nhất ta có thể chia giai đoạn theo thứ tự tô pô của nó.

Xét một ví dụ cụ thể.

Ví dụ 3. Bài toán người bán hàng Euclid yêu cầu, với n điểm cho trước trên mặt phẳng, xác định một hành trình khép kín đi qua các điểm ấy. Hình 5(a) cho lời giải của bài toán bảy điểm. Bài toán hành trình bitonic là một dạng đơn giản hơn của bài toán người bán hàng Euclid: hành trình xuất phát từ điểm ngoài cùng bên trái, đi nghiêm ngặt từ trái sang phải đến điểm ngoài cùng bên phải, rồi lại đi nghiêm ngặt từ phải sang trái về điểm xuất phát; cần tìm độ dài đường đi ngắn nhất. Hình 5(b) cho lời giải của bài toán bảy điểm.

Hai bài toán này nhìn có vẻ rất giống nhau, nhưng về bản chất lại khác nhau. Để tiện thảo luận, ta đánh số các đỉnh. Vì hành trình chắc chắn đi qua đỉnh ngoài cùng bên phải, là đỉnh 7, nên một đường $(P_1 - P_2)$ có thể xem là sự kết hợp của hai đường $(P_1 - 7)$ và $(P_2 - 7)$. Do đó trạng thái của bài này có thể biểu diễn bằng dạng kết hợp của hai đường. Ta có thể xếp các trạng thái có cùng hai đỉnh xuất phát của hai đường vào một giai đoạn, ký hiệu là giai đoạn $[P_1, P_2]$.

Đối với bài toán hành trình bitonic, nếu giai đoạn $[P_1, P_2]$ có thể được suy ra từ giai đoạn $[Q_1, Q_2]$, thì điều kiện bắt buộc là $P_1 < Q_1$ hoặc $P_2 < Q_2$. Ví dụ, đường trong giai đoạn $[3, 4]$ có thể thu được từ đường trong giai đoạn $[3, 5]$ bằng cách thêm cạnh $4 - 5$; nhưng trạng thái của giai đoạn $[3, 5]$ lại không thể thu được từ trạng thái của giai đoạn $[3, 4]$, vì hành trình bitonic yêu cầu phải đi nghiêm ngặt từ trái sang phải. Do đó nếu ta đã biết trạng thái trong giai đoạn $[3, 4]$, thì trạng thái trong giai đoạn $[3, 5]$ tất yếu đã biết. Vì vậy, ta nói bài toán bitonic thỏa nguyên tắc không hậu hiệu và có thể giải bằng quy hoạch động; chương trình xem Pro_3_1.Pas.

Đối với bài toán người bán hàng Euclid, giữa các giai đoạn không có một thứ tự tất yếu nào. Chẳng hạn đường $\{3 - 2 - 5 - 7, 4 - 6 - 7\}$ thuộc giai đoạn $[3, 4]$, có thể suy ra từ đường $\{2 - 5 - 7, 4 - 6 - 7\}$ thuộc giai đoạn $[2, 4]$; còn đường $\{2 - 3 - 6 - 7, 4 - 5 - 7\}$ thuộc giai đoạn $[2, 4]$, có thể suy ra từ đường $\{3 - 6 - 7, 4 - 5 - 7\}$ thuộc giai đoạn $[3, 4]$. Nếu dùng đỉnh để biểu diễn giai đoạn và dùng quan hệ suy ra để biểu diễn cạnh, thì quan hệ tương ứng giữa hai giai đoạn $[3, 4]$ và $[2, 4]$ giống Hình 6. Ta thấy rất rõ rằng quan hệ giữa hai giai đoạn này có hậu hiệu, vì trong đồ thị có chu trình. Với bài toán này, ta không thể giải như bài trước. Thực tế, đây là một bài toán NP-đầy đủ, thời gian giải rất có khả năng là cấp mũ.

Vì vậy, một điều kiện phán đoán rất quan trọng để xem một bài toán có thể giải bằng quy hoạch động hay không là: nó có hậu hiệu hay không. Khi phán đoán điều này, một cách hiệu quả là xem các giai đoạn của bài toán như đỉnh, quan hệ giữa các giai đoạn như cạnh có hướng, rồi kiểm tra đồ thị có hướng ấy có phải là đồ thị không chu trình hay không, tức có thể sắp thứ tự tô pô hay không.

Từ các phân tích trên, ta có thể rút ra một số phương pháp và bước cơ bản để giải bài quy hoạch động:

1. Xác định đối tượng nghiên cứu của bài toán, tức xác định trạng thái.
2. Chia giai đoạn, xác định phương trình chuyển trạng thái giữa các giai đoạn.
3. Xem bài toán hiện tại có thể giải bằng quy hoạch động hay không: kiểm tra bài toán có cấu trúc con tối ưu hay không, và có tính không hậu hiệu hay không.

4. Nếu phát hiện bài toán hiện tại chưa thể giải bằng quy hoạch động, cần điều chỉnh các định nghĩa và cách chia tương ứng để đạt đến dạng có thể giải bằng quy hoạch động.

Trên đây chỉ là quá trình tư duy quy hoạch động trong trường hợp thông thường. Một số bài đơn giản có thể thao tác tuần tự theo các bước ấy, nhưng đa số bài quy hoạch động, đặc biệt là bài trong thi tin học, kiểm tra nhiều mảng kiến thức và có kỹ thuật vận dụng linh hoạt. Dưới đây ta thảo luận một số điểm trọng yếu và khó trong ứng dụng quy hoạch động.

3 Ứng dụng thực tế của quy hoạch động

Khi đánh giá một thuật toán, tiêu chuẩn chủ yếu không ngoài hai chỉ tiêu: thời gian và không gian. Thời gian của đa số thuật toán quy hoạch động là cấp đa thức; so với thời gian cấp mũ của thuật toán tìm kiếm giải cùng bài toán, thời gian của quy hoạch động là rất ít. Vì vậy trong ứng dụng thực tế, ta hiếm khi cân nhắc vấn đề thời gian của quy hoạch động; vấn đề thường gặp nhất là không gian, tức chọn và lưu trữ trạng thái.

3.1 Chọn trạng thái

Đối với một thuật toán quy hoạch động, việc chia giai đoạn không quá quan trọng, vì giai đoạn chỉ là tập các trạng thái có thể xử lý giống nhau; ta hoàn toàn có thể định nghĩa mỗi trạng thái riêng lẻ là một giai đoạn. Vì vậy, việc chọn trạng thái đóng vai trò quyết định đối với toàn bộ cách xử lý bài toán. Trạng thái được chọn phải thỏa hai điểm sau:

1. Trạng thái phải mô tả đầy đủ tính chất của sự vật; hai sự vật khác nhau phải có trạng thái khác nhau.
2. Phải tồn tại phương trình chuyển giữa các trạng thái, để ta có thể chuyển dần từ trạng thái ứng với bài toán ban đầu đến trạng thái ứng với bài toán kết thúc.

Trạng thái là đại lượng mô tả tính chất của sự vật, vì vậy ta cần dựa vào tiêu chuẩn này và phân tích cụ thể theo yêu cầu của đề bài. Xét ví dụ sau.

Ví dụ 4. Có một công ty vận chuyển bò, cần vận chuyển bò giữa bảy nông trại A, B, C, D, E, F, G . Xe xuất phát từ A , sau khi hoàn thành nhiệm vụ phải quay về A . Mỗi lần xe chỉ chở được một con bò. Mỗi nhiệm vụ vận chuyển được cho bởi một cặp chữ cái, lần lượt biểu thị bò được chuyển từ nông trại nào đến nông trại nào. Tổng số nhiệm vụ là N , $1 \leq N \leq 12$. Biết khoảng cách giữa các nông trại, hãy tìm một lộ trình ngắn nhất để hoàn thành mọi nhiệm vụ.

Ta phân tích cách giải. Mục tiêu của ta là hoàn thành một số nhiệm vụ. Vì xe mỗi lần chỉ chở được một con bò, ta chỉ có thể xử lý các nhiệm vụ lần lượt. Do đề bài yêu cầu lộ trình ngắn nhất để hoàn thành các nhiệm vụ, nên trong quá trình hoàn thành nhiệm vụ, mọi đoạn đường ta đi đều là đường ngắn nhất. Nói cách khác, một khi đã xác định thứ tự hoàn thành nhiệm vụ $P_1 - P_2 - \dots - P_N$, thì thời gian ngắn nhất của lộ trình ấy cũng xác định. Mục tiêu của ta là xác định thứ tự hoàn thành N nhiệm vụ. Vì vậy, đối tượng nghiên cứu của bài toán là một tập nhiệm vụ S .

Vậy trạng thái nên định nghĩa thế nào? Vì đối tượng nghiên cứu là tập hợp, ta tự nhiên nghĩ đến cách mô tả quan hệ phần tử trong tập. Có thể định nghĩa trạng thái là một mảng $(t_1, t_2, \dots, t_N)$, trong đó mỗi t_i bằng 0 nếu nhiệm vụ i không thuộc tập hiện tại, và bằng 1 nếu nhiệm vụ i thuộc tập hiện tại. Nhưng nếu định nghĩa như vậy, ta sẽ không viết được phương trình chuyển trạng thái, vì còn liên quan đến độ dài đường ngắn nhất ở “điểm nối” giữa hai nhiệm vụ trước sau. Vì thế ta cần thêm một đại lượng x để giới hạn nhiệm vụ đầu tiên cần thực hiện trong tập nhiệm vụ hiện tại. Một trạng thái được định nghĩa là

$$(x, t_1, t_2, \dots, t_N),$$

trong đó t_x bắt buộc bằng 1. Trọng số ứng với trạng thái là độ dài lộ trình ngắn nhất để hoàn thành tập nhiệm vụ do trạng thái mô tả. Phương trình chuyển là

$$(x, t_1, t_2, \dots, t_N) = \min\{(y, p_1, p_2, \dots, p_N) + \text{Dis}(x, y)\}.$$

Trong đó $y \neq x$ và $t_y = 1$; nếu $i \neq x$ thì $p_i = t_i$, còn $p_x = 0$. Giả sử $\text{Dis}(x, y)$ là khoảng cách ngắn nhất từ thành phố kết thúc của nhiệm vụ x đến thành phố bắt đầu của nhiệm vụ y . Điều kiện đầu của chuyển trạng thái là

$$(x, t_1, t_2, \dots, t_N) = \{\text{khoảng cách từ thành phố kết thúc của nhiệm vụ } x \text{ đến } A\},$$

trong đó $t_x = 1$, các t_i còn lại đều bằng 0.

Trong xử lý thực tế, dùng một mảng tối đa 12 chiều như vậy rõ ràng không tiện. Vì mỗi bit chỉ có thể là 0 hoặc 1, ta xem $(t_1, t_2, \dots, t_N)$ là biểu diễn nhị phân của một số thập phân. Khi đó mảng trạng thái là một mảng hai chiều. Chương trình cụ thể xem Pro_4_1.Pas trong phụ lục.

Từ bài toán này có thể thấy, chọn trạng thái trong quy hoạch động thực chất là chọn cách mô tả thích đáng và ngắn gọn nhất cho sự vật trong bài toán. Việc chọn trạng thái không thể hoàn thành trong một bước, mà là quá trình dần điều chỉnh và hoàn thiện trong khi suy nghĩ.

3.2 Lưu trữ trạng thái

Sau khi trạng thái đã được chọn, vấn đề tiếp theo là lưu trữ trạng thái. Về lý thuyết, mỗi trạng thái đều nên lưu hai giá trị: trọng số tối ưu của trạng thái ấy và giá trị đánh dấu quyết định. Nhưng trên thực tế, nhiều bài quy hoạch động có số trạng thái rất lớn; muốn chương trình hóa thuật toán, ta phải tối ưu lưu trữ. Tối ưu chủ yếu là bỏ mọi lượng lưu trữ không cần thiết.

Trong một số bài, đề bài chỉ cho quan hệ trong một phạm vi nào đó, nên ta chỉ cần lưu các trạng thái trong phạm vi ấy. Xét bài toán sau.

Ví dụ 5. Cấu trúc của một sinh vật có thể biểu diễn bằng một dãy “nguyên tố”, và một nguyên tố được biểu diễn bằng một chuỗi chữ cái tiếng Anh. Với một tập nguyên tố P , có thể tạo chuỗi S bằng cách nối lần lượt N nguyên tố $P_1, P_2, \dots, P_N$. Bài toán là: cho chuỗi S và tập nguyên tố P , tìm tiền tố dài nhất của S có thể tạo bởi các nguyên tố trong P . Độ dài lớn nhất của mỗi nguyên tố là 20, độ dài lớn nhất của chuỗi là 500000. Ví dụ tập nguyên tố là $\{A, AB, BBC, CA, BA\}$, chuỗi là ABABACABAABCB, thì độ dài lớn nhất là 11; cách ghép cụ thể xem Hình 7, trong đó các nguyên tố khác nhau được tô màu khác nhau.

Trạng thái của bài toán này rất dễ xác định. Mỗi vị trí trong chuỗi chỉ có hai tính chất: tiền tố của chuỗi kết thúc tại vị trí ấy có thể được tạo bởi dãy nguyên tố hay không. Vì vậy ta đặt một mảng

Could: Array[1..Max] Of Boolean

để biểu diễn trạng thái của từng vị trí. Phương trình chuyển là

$$\begin{aligned} \text{Could}[K] &:= \text{Could}[K] \text{ Or } (\text{Could}[K - W] \text{ And} \\ &\quad S[K - W + 1 \dots K] = J_i[I]). \end{aligned}$$

Trong đó giá trị đầu của $\text{Could}[k]$ đều là False; $J_i[I]$ biểu diễn chuỗi ứng với nguyên tố thứ I , độ dài là W ; $S[K - W + 1 \dots K]$ là chuỗi gồm các ký tự từ vị trí $K - W + 1$ đến K trong chuỗi đã cho.

Đến đây, bài toán dường như đã được giải quyết: chỉ cần tìm I lớn nhất sao cho $\text{Could}[I]$ là True. Nhưng quay lại nhìn đề bài, ta thấy chuỗi dài nhất là 500000, căn bản không thể lưu trữ theo cách thô. Phải làm gì? Quan sát kỹ phương trình chuyển, ta thấy $\text{Could}[I]$ chỉ liên quan đến $\text{Could}[I - W]$, trong

khi W là độ dài nguyên tố và độ dài nguyên tố lớn nhất là 20. Nói cách khác, muốn suy ra $\text{Could}[I]$, ta nhiều nhất chỉ cần biết các giá trị từ $\text{Could}[I - 20]$ đến $\text{Could}[I - 1]$. Các giá trị khác có thể bỏ qua. Như vậy số giá trị Could cần ghi giảm đột ngột từ 500000 xuống 20; thay đổi này là rất lớn. Chương trình cụ thể xem `Pro_5.Pas`.

Vì sao bài toán này chỉ cần ghi rất ít trạng thái? Đó là do phương trình chuyển và thứ tự giải đặc biệt của nó. Thứ tự giải được quyết định theo thứ tự vị trí trong chuỗi, còn trong phương trình chuyển, trạng thái hiện tại chỉ liên quan đến 20 trạng thái đã giải trước nó. Vì vậy bài toán này có thể xử lý như trên.

Nhưng với đa số bài toán, cấu trúc đẹp như vậy chưa chắc tồn tại. Khi giải những bài ấy, ta chỉ có thể cố hết sức giảm lượng lưu trữ. Xét ví dụ sau.

Ví dụ 6. Một công ty nhập vào một lô thùng, tổng số là N , $1 \leq N \leq 1000$, được đưa vào lần lượt bằng băng chuyền, rồi xếp trong kho thành nhiều nhất P cột, $1 \leq P \leq 4$, như Hình 8.

Biết rằng các thùng nhiều nhất thuộc M loại, $1 \leq M \leq 20$. Người ta muốn các thùng cùng loại được xếp gần nhau nhất có thể để đẹp mắt. Độ đẹp T được định nghĩa là:

$$T = \sum (\text{số loại khác nhau nhìn thấy lần lượt trong mỗi cột}).$$

“Số loại khác nhau nhìn thấy lần lượt” nghĩa là nếu thùng thứ K trong một cột khác loại với thùng thứ $K - 1$, thì giá trị độ đẹp tăng thêm 1. Hãy tìm một cách điều phối sao cho tổng độ đẹp của các cột là nhỏ nhất.

Ý tưởng của bài toán như sau. Thùng thứ K cần chọn một cột để đặt. Nếu nó được đặt lên một thùng khác loại, nó làm tổng độ đẹp tăng thêm 1. Nói cách khác, vị trí đặt thùng thứ K chỉ liên quan đến tình hình ở đỉnh của P cột hiện tại. Nếu muốn tìm giá trị nhỏ nhất sau khi đặt thùng này, tự nhiên ta cần giá trị nhỏ nhất của $K - 1$ thùng trước đó trong tình hình đỉnh cột tương ứng. Bài toán quyết định nhiều giai đoạn này đồng thời thỏa nguyên lý tối ưu và nguyên tắc không hậu hiệu, nên có thể giải bằng quy hoạch động.

Cách chọn trạng thái rất rõ ràng: (k, P_1, P_2, P_3, P_4) . K là số hiệu của thùng sắp được đặt; P_1 đến P_4 là tình hình đỉnh của mỗi cột sau khi đặt thùng thứ K , tức loại thùng đang nằm trên đỉnh mỗi cột, giả sử hiện có bốn cột. Cách chia giai đoạn cũng rõ ràng: chia theo thứ tự thùng đi tới từ băng chuyền. Phương trình chuyển trạng thái là

$$(K, P_1, P_2, P_3, P_4) = \min(K - 1, Q_1, Q_2, Q_3, Q_4) + \text{Dis}.$$

Trong đó Q_1 đến Q_4 là tình hình các cột trước khi đặt thùng thứ K , còn Dis bằng 0 hoặc 1, tùy thùng thứ K có khác loại với thùng ngay dưới nó trong cột hay không.

Đến đây, mọi bước của thuật toán quy hoạch động đã rõ, dường như có thể viết chương trình trực tiếp. Nhưng với bài này, lưu trữ một lượng lớn thông tin là vấn đề bắt buộc phải đối diện. Không gian khả dụng lớn nhất mà Pascal cho phép là 640KB; trong chế độ bảo vệ, nhiều nhất cũng chỉ hơn 1000KB. Một chương trình quy hoạch động thường cần lưu hai giá trị: thông tin quyết định và giá trị trạng thái. Với bài này, nếu làm theo cách thông thường, lúc nhiều nhất cần $1000 \times 20^3 \times (1 + 2)$ byte, hoàn toàn không thể lưu. Ta phải tối ưu lưu trữ và bỏ mọi lượng không cần thiết:

1. Chỉ ghi thông tin quyết định ở mỗi bước, không ghi giá trị trạng thái ở mỗi bước; nhờ vậy có thể tiết kiệm $\frac{2}{3}$ không gian.
2. Với trạng thái ở mỗi bước, P đỉnh cột tương ứng không chứa hai thùng cùng loại.
3. Ngoài cột được chọn ở bước này, thứ tự sắp xếp của $P - 1$ đỉnh cột còn lại không ảnh hưởng đến kết quả; vì vậy trạng thái cần lưu là tổ hợp của $P - 1$ cột còn lại, không phải hoán vị.

Như vậy tổng lượng lưu trữ có thể giảm từ 24000KB xuống 1200KB, giảm khoảng 20 lần, và hoàn toàn thao tác được trong chế độ bảo vệ của Pascal. Chương trình cụ thể xem Pro_6.Pas.

Qua hai bài trên, ta thấy trong ứng dụng thực tế, tốc độ của quy hoạch động hầu như không thành vấn đề; nhưng khâu lưu trữ trạng thái thì phải suy nghĩ đi suy nghĩ lại, tối ưu rồi tối ưu nữa. Trong các đề thi vài năm gần đây, khó điểm và trọng điểm của đa số bài quy hoạch động chính là chọn và lưu trữ trạng thái. Đây cũng là trọng điểm hàng đầu trong ứng dụng thực tế của quy hoạch động.

4 Suy nghĩ sâu hơn về quy hoạch động

Trên đây đã thảo luận nền tảng lý thuyết của quy hoạch động và các vấn đề có thể gặp trong ứng dụng thực tế. Vậy quy hoạch động tốt ở đâu? Tại sao ta cần dùng quy hoạch động? Từ các ví dụ trên có thể thấy ưu điểm lớn nhất của quy hoạch động là tiết kiệm thời gian. Ưu điểm rất lớn này có thể thấy từ cách giải Ví dụ 1. Trong thực tế, thời gian của thuật toán tìm kiếm và thuật toán quy hoạch động khác nhau rất lớn. Bảng 1 là thời gian chạy của hai chương trình tương ứng với Ví dụ 1.

Dữ liệu thử	1	2	3	4	5
Tìm kiếm	1.04s	1.42s	1.05s	7.42s	22.92s
Quy hoạch động	< 0.01s	< 0.01s	0.06s	0.02s	0.01s

Bảng 1

Từ bảng thời gian chạy này có thể thấy, thời gian chạy của quy hoạch động và tìm kiếm trong ứng dụng thực tế khác nhau rất lớn. Về lý thuyết, độ phức tạp thời gian của tìm kiếm là $O(N!)$, còn độ phức tạp thời gian của quy hoạch động là $O(N^2)$; hai thời gian này căn bản không cùng một cấp. Tăng trưởng thời gian cấp mũ là rất đáng sợ, lớn hơn rất nhiều so với thuật toán cấp đa thức. Vì vậy, ta thường gọi thuật toán cấp đa thức là thuật toán “khá tốt”, và thuật toán cấp mũ là thuật toán “khá kém”.

Về bản chất, chương trình quy hoạch động là một kỹ thuật dùng chi phí không gian để đổi lấy chi phí thời gian. Trong quá trình cài đặt, nó buộc phải lưu các trạng thái sinh ra. Vì vậy, độ phức tạp không gian của nó lớn hơn các thuật toán khác. Lấy bài toán hành trình bitonic làm ví dụ: bài này cũng có thể giải bằng tìm kiếm, xem chương trình Pro_3_2.Pas. Độ phức tạp thời gian của quy hoạch động là $O(N^2)$, của tìm kiếm là $O(N!)$; nhưng xét độ phức tạp không gian, quy hoạch động là $O(N^2)$, còn tìm kiếm là $O(N)$. Tìm kiếm lại tốt hơn quy hoạch động về không gian. Vậy vì sao ta chọn quy hoạch động để giải Ví dụ 3? Vì không gian của quy hoạch động có thể chịu được, thực tế là rất dễ chịu, còn thời gian của tìm kiếm thì quá lớn. Vì vậy ta bỏ không gian để lấy thời gian.

Điều này còn rõ hơn trong Ví dụ 4, tức bài toán vận chuyển bờ. Về bản chất, Ví dụ 4 là bài toán tìm chu trình Hamilton tối ưu. Đây là một bài toán NP; đến nay người ta vẫn chưa tìm được lời giải cấp đa thức cho nó. Vì vậy, khi dùng quy hoạch động, ta buộc phải ghi lại từng trạng thái rồi truy hồi. Khi truy hồi, dường như ta đang thực hiện một thuật toán cấp đa thức. Thực chất, số trạng thái đã ghi lại vốn đã là cấp mũ. Thuật toán quy hoạch động của ta, về lý thuyết, có độ phức tạp thời gian gần như thuật toán tìm kiếm thông thường, còn độ phức tạp không gian lại cao hơn tìm kiếm thông thường rất nhiều: không gian của quy hoạch động là $O(2^N)$, của tìm kiếm là $O(N)$. Vậy vì sao dùng quy hoạch động cho bài này lại nhanh hơn tìm kiếm? Nhanh hơn bao nhiêu có thể xem Bảng 2. Phần không gian dùng thêm rất cuộc để làm gì? Câu trả lời cho hai câu hỏi này chính là mục đích căn bản của kỹ thuật quy hoạch động: xử lý dư thừa.

Trong quá trình dùng tìm kiếm, ta làm một số việc đã từng làm. Lấy Ví dụ 4: giả sử tập nhiệm vụ hiện cần giải là [1, 2, 3, 4]. Nếu ta chọn hai nhiệm vụ đầu tiên cần giải là 1, 2, thì còn phải chọn thứ tự giải từ hai nhiệm vụ sau là 3, 4. Nếu ta chọn hai nhiệm vụ đầu tiên là 2, 1, khác với 1, 2 vì thứ tự giải khác nhau, thì lại phải giải lại công việc “chọn thứ tự giải từ hai nhiệm vụ sau 3, 4”. Lãng phí lớn biết

bao! Câu này đã được nhắc một lần khi phân tích Ví dụ 1 ở đầu bài; nhắc lại ở đây nhằm vạch rõ tư duy cơ bản của kỹ thuật quy hoạch động: không làm lại việc đã làm.

Với những việc đã làm, ta đã biết kết quả của chúng. Khi gặp lại cùng một công việc lần thứ hai, hoàn toàn không cần làm lại; chỉ cần lấy kết quả lần trước ra dùng. Ý tưởng “lười biếng” này chính là thái độ căn bản của con người khi nghiên cứu vấn đề trong suốt hàng nghìn năm. Các định lý, định luật trong toán học, vật lý và các ngành tự nhiên mà ta thường gặp, về bản chất, chẳng phải đều là “bản ghi công việc” hay sao? Mục đích học các định lý, định luật ấy một mặt là học phương pháp suy nghĩ, mặt khác chẳng phải là để lần sau gặp những vấn đề đó thì giải trực tiếp hay sao? Vì vậy, kỹ thuật quy hoạch động thực chất là sự tiếp nối của ý tưởng ấy trong lĩnh vực máy tính, một sự bộc lộ tự nhiên.

Mục đích của việc giải bài là để lần sau không phải giải bài nữa. Nói theo một câu cổ, điều ta hy vọng cuối cùng là: khi giải bài có thể “lấy cái bất biến ứng với vạn biến”, vì vạn biến vẫn không rời khỏi gốc, và bài toán “gốc” ấy ta đã từng giải; lần này chỉ cần đọc dữ liệu và đọc kết quả.

Hiểu rõ đạo lý này, năng lực giải bài của ta sẽ tiến thêm một bậc. Với Ví dụ 4, ta hoàn toàn có thể dùng thuật toán tìm kiếm, đồng thời dùng một mảng ghi lại mọi trạng thái để hỗ trợ tìm kiếm: một khi phát hiện nhiệm vụ tìm kiếm này đã từng được tìm, ta trực tiếp đọc giá trị tối ưu. Như vậy không còn dư thừa, thời gian của thuật toán tự nhiên sẽ tốt như quy hoạch động, thậm chí còn tốt hơn vì trong tìm kiếm có thêm cắt nhánh giữa chừng. Xem Bảng 2.

Dữ liệu thử	1	2	3	4	5
Quy hoạch động	0.11s	0.09s	0.22s	0.22s	0.60s
Tìm kiếm	1.58s	1.90s	5.27s	6.58s	7.09s
Tìm kiếm cải tiến	0.06s	0.01s	0.05s	0.07s	0.16s

Bảng 2

Từ đây ta phát hiện: tìm kiếm chậm vì dư thừa; sau khi bỏ phần dư thừa, công việc trở nên đơn giản hơn nhiều. Tạm thời không xét yếu tố có thể cắt nhánh giữa chừng, tức mọi công việc trong quá trình tìm kiếm đều có thể sinh ra kết quả ta cần. Khi đó những công việc này là bắt buộc, mọi thuật toán đều phải làm, không thể thiếu, và cũng là giới hạn công việc để giải bài toán tổng thể. Vì không làm thêm bất kỳ việc nào, tốc độ giải bài của ta sao có thể không nhanh?

Đó là nguyên nhân căn bản khiến quy hoạch động có tốc độ đáng kinh ngạc: nếu không đổi hướng suy nghĩ, khối lượng công việc ta làm sẽ không ít hơn giới hạn ấy: xét mỗi trạng thái khả dĩ đúng một lần.

5 Kết luận

Qua các thảo luận trên, ta biết rằng quy hoạch động là một kỹ thuật có hiệu suất rất cao. Thời gian chạy nhỏ là vì nó lưu kết quả của những công việc đã làm. Đa số bài toán quyết định nhiều giai đoạn hầu như đều có thuật toán quy hoạch động; chỉ có một số bài bắt buộc phải ghi mọi trạng thái, gây căng thẳng về không gian, nên cuối cùng phải từ bỏ quy hoạch động.

Vấn đề thời gian gần như không thành vấn đề đối với thuật toán quy hoạch động, vì thuật toán quy hoạch động đã đạt tới giới hạn thời gian. Nhưng vấn đề không gian lại là vấn đề lớn nhất khi ứng dụng kỹ thuật quy hoạch động. Lượng lưu trữ trạng thái khổng lồ là điểm đau đầu nhất khi dùng quy hoạch động, và tự nhiên trở thành một điểm kiểm tra quy hoạch động trong thi tin học. Chọn trạng thái thế nào, tối ưu lưu trữ trạng thái ra sao, lại càng là vấn đề mỗi người có cách nhìn riêng. Cuối cùng, tôi tổng kết các bước đại khái để giải bài quy hoạch động:

1. **Suy nghĩ (Think):** nghĩ ra phương pháp giải bài. Bước này đặt trên cơ sở hiểu và phân tích bài toán;

thông qua việc nghiền ngẫm cẩn thận bài toán, phân biệt nghiêm túc ưu khuyết của từng hướng, rồi chọn hướng hiệu quả và thích đáng nhất để giải bài.

2. **Lập kế hoạch** (Design): xác định các điều kiện của quy hoạch động: trạng thái, giai đoạn, quyết định, phương trình chuyển, điều kiện biên. Đây là bước điều phối toàn cục, đặt nền tảng thuật toán quy hoạch động và điều kiện áp dụng cho bài toán.
3. **Thực hiện** (Operate): chuyển thuật toán thành chương trình. Kết hợp với điều kiện máy tính và môi trường lập trình để giải quyết cụ thể vấn đề lưu trữ trạng thái, từ đó giải toàn bộ bài toán. Bước này được hoàn thành nhờ sự phối hợp giữa con người và máy tính.

Theo các bước trên, một chương trình quy hoạch động về cơ bản đã được hoàn thành.

Suy cho cùng, quy hoạch động chỉ là một kỹ thuật, cần được hoàn thiện và phát triển qua ứng dụng liên tục. Chỉ thông qua không ngừng ứng dụng và suy nghĩ, ta mới có thể hiểu rõ cấu trúc của nó, lĩnh hội nội hàm của nó, và thật sự nắm vững kỹ thuật này.

A Phụ lục

Bài viết có 6 ví dụ. Đề bài trong phần chính đều được viết ngắn; ở đây đưa ra mô tả chi tiết và chương trình giải cho từng ví dụ. Các hình được nhắc đến trong đề đã xuất hiện ở phần chính, với cùng số hiệu.

A.1 Ví dụ 1

Đề bài. Hình 1 cho một mạng đường đi. Số trên đường nối giữa hai điểm biểu diễn khoảng cách hoặc chi phí giữa hai điểm. Hãy tìm một lộ trình từ *A* đến *E* sao cho tổng khoảng cách hoặc tổng chi phí nhỏ nhất.

Ghi chú. Bài này lấy từ *Hướng dẫn thi Olympic tin học: thuật toán và thiết kế chương trình trong tổ hợp*. Trong phần chính, để so sánh hiệu suất của tìm kiếm và quy hoạch động, bài viết đưa ra hai chương trình Pro_1_1.Pas và Pro_1_2.Pas, cùng năm dữ liệu thử Data_1_i.Txt và đáp án tương ứng Sol_1_i.Txt trong thư mục Data.

Chương trình.

```
Program Pro_1_1;
```

```
Const
```

```
Max      =300;  
Inputfile  ='Input.Txt';  
Outputfile  ='Ouput.Txt';  
Big       =1000000;
```

```
Type
```

```
Maps      =Array [1..Max] Of Integer;
```

```
Var
```

```
Se        :Set Of Byte;  
Map       :Array[1..Max] Of ^maps;
```

```

Qiu,Pr      :Array[1..Max] Of Byte;
F           :Text;
Lp          :Integer;
N,M         :Integer;
Min         :Longint;

Procedure Init;
Var
  I,J,K,W   :Integer;
Begin
  Assign(F,Inputfile);
  Reset(F);
  Readln(F,N,M);
  For I:=1 To N Do
  Begin
    New(Map[I]);
    Fillchar(Map[I]^,Sizeof(Map[I]^),0);
  End;
  For I:=1 To M Do
  Begin
    Readln(F,J,K,W);
    Map[J]^[K]:=W;
  End;
  Close(F);
End;

Procedure Search(Dep:Byte;Al:Longint);
Var
  I         :Byte;
Begin
  If Al>=Min Then Exit;
  If Qiu[Dep-1]=N Then
  Begin
    Min:=Al;
    Pr:=Qiu;
    Lp:=Dep-1;
  End
  Else
  For I:=2 To N Do
    If (I In Se) And (Map[Qiu[Dep-1]]^[I]>0) Then
    Begin
      Exclude(Se,I);
      Qiu[Dep]:=I;
      Search(Dep+1,Al+Map[Qiu[Dep-1]]^[Qiu[Dep]]);
      Include(Se,I);
    End;
  End;
End;

```

```

Procedure Print;
Var
    I          :Integer;
Begin
    Assign(F,Outputfile);
    Rewrite(F);
    Writeln(F,Min);
    For I:=1 To Lp Do
        Write(Pr[I],' ');
    Close(F);
End;

Begin
    Init;
    Se:=[2..N];
    Qiu[1]:=1;
    Min:=Big;
    Search(2,0);
    Print;
End.

Program Pro_1_2;

Const
    Max          =300;
    Inputfile    ='input.Txt';
    Outputfile   ='output.Txt';
    Big          =1000000;

Type
    Maps         =Array[1..Max] Of Integer;

Var
    Se           :Set Of Byte;
    Map          :Array[1..Max] Of ^maps;
    Dis          :Array[1..Max] Of Longint;
    Fr           :Array[1..Max] Of Byte;
    Bo           :Array[1..Max] Of Boolean;
    N,M          :Integer;
    F            :Text;

Procedure Init;
Var
    I,J,K,W     :Integer;
Begin
    Assign(F,Inputfile);
    Reset(F);
    Readln(F,N,M);

```

```

For I:=1 To N Do
Begin
    New(Map[I]);
    Fillchar(Map[I]^,Sizeof(Map[I]^),0);
End;
For I:=1 To M Do
Begin
    Readln(F,J,K,W);
    Map[J]^[K]:=W;
End;
Close(F);
End;

Procedure Main;
Var
    I,J,Who :Integer;
    Min      :Longint;
Begin
    Dis[N]:=0;
    Who:=N;
    Fillchar(Fr,Sizeof(Fr),0);
    Fillchar(Bo,Sizeof(Bo),True);
    Bo[N]:=False;
    While Who<>1 Do
    Begin
        For I:=1 To N Do
            If Map[I]^[Who]>0 Then
                If (Fr[I]=0) Or (Dis[I]>Dis[Who]+Map[I]^[Who]) Then
                    Begin
                        Dis[I]:=Dis[Who]+Map[I]^[Who];
                        Fr[I]:=Who;
                    End;
        Min:=Big;
        For I:=1 To N Do
            If Bo[I] And (Fr[I]>0) And (Dis[I]<Min) Then
                Begin
                    Who:=I;
                    Min:=Dis[I];
                End;
        Bo[Who]:=False;
    End;
End;

Procedure Print;
Var
    I      :Integer;
Begin
    Assign(F,Outputfile);

```

```

Rewrite(F);
Writeln(F,Dis[1]);
I:=1;
While I<>N Do
Begin
    Write(F,I,' ');
    I:=Fr[I];
End;
Writeln(F,N);
Close(F);
End;

Begin
    Init;
    Main;
    Print;
End.

```

A.2 Ví dụ 2

Đề bài. Có bốn điểm A, B, C, D , như Hình 4. Giữa hai điểm kề nhau có hai đường nối C_{2k}, C_{2k-1} ($1 \leq k \leq 3$), biểu diễn hai đường có thể đi. Số trên đường nối biểu diễn độ dài đường. Ta định nghĩa trong mọi đường từ A đến D , đường có số dư nhỏ nhất khi độ dài chia cho 4 là đường tối ưu.

Ghi chú. Bài này lấy từ tạp chí *Olympic tin học*, số 1–2 năm 1998.

A.3 Ví dụ 3

Đề bài. Bài toán người bán hàng Euclid là bài toán xác định một hành trình khép kín liên kết N điểm cho trước trên mặt phẳng. Hình 5(a) cho lời giải của bài toán bảy điểm. Dạng tổng quát của bài toán này là NP-đầy đủ, do đó lời giải cần nhiều hơn thời gian đa thức.

J. L. Bentley đề nghị đơn giản hóa bằng cách chỉ xét hành trình bitonic: xuất phát từ điểm trái nhất, đi nghiêm ngặt từ trái sang phải đến điểm phải nhất, rồi nghiêm ngặt từ phải sang trái về điểm xuất phát. Hình 5(b) biểu diễn hành trình bitonic ngắn nhất của cùng bảy điểm. Trong trường hợp này, một thuật toán thời gian đa thức là có thể.

Hãy mô tả một thuật toán $O(N^2)$ để xác định hành trình bitonic tối ưu. Có thể giả sử mọi cặp nút đều có hoành độ khác nhau.

Ghi chú. Bài này lấy từ *Đại toàn tập cấu trúc dữ liệu máy tính và thuật toán thực dụng*. Bài viết trình bày hai cách giải: quy hoạch động và tìm kiếm.

Chương trình.

```

Program Pro_3_1;
Const
    Inputfile  = 'input.Txt';
    Outputfile = 'Output.Txt';

```

```

Max      =100;
Big      =10000;

Var
  F      :Text;
  Po      :Array[1..Max,1..2] Of Integer;
  Dis     :Array[1..Max,1..Max] Of Real;
  N      :Integer;

Procedure Init;
Var
  I      :Integer;
Begin
  Assign(F,Inputfile);
  Reset(F);
  Readln(F,N);
  For I:=1 To N Do
    Readln(F,Po[I,1],Po[I,2]);
  Close(F);
End;

Function Len(P1,P2:Integer):Real;
Begin
  Len:=Sqrt(Sqr(Po[P1,1]-Po[P2,1])+Sqr(Po[P1,2]-Po[P2,2]));
End;

Procedure Main;
Var
  I,J,K :Integer;
  Now   :Real;
Begin
  Dis[N,N]:=0;
  For I:=N-1 Downto 1 Do
    Begin
      Dis[I,N]:=Len(I,I+1)+Dis[I+1,N];
      Dis[N,I]:=Dis[I,N];
    End;

  For I:=N-2 Downto 1 Do
    For J:=N-1 Downto I+1 Do
      Begin
        If I+1<J Then Now:=Dis[I+1,J]+Len(I,I+1)
        Else
          Begin
            Now:=Big;
            For K:=J+1 To N Do
              If Dis[K,J]+Len(I,K)<Now Then
                Now:=Dis[K,J]+Len(I,K);
          End;
      End;
    End;
  End;

```

```

        End;
    For K:=J+1 To N Do
        If Dis[I,K]+Len(J,K)<Now Then
            Now:=Dis[I,K]+Len(J,K);
        Dis[I,J]:=Now;
        Dis[J,I]:=Now;
    End;
End;

Procedure Print;
Var
    X1,X2,I,D :Integer;
    Now        :Real;
    P          :Array[1..2] Of Integer;
    Pr         :Array[1..2,1..Max] Of Byte;
    Ok         :Boolean;

    Procedure Change;
    Var
        G      :Integer;
    Begin
        D:=3-D;
        G:=X1;X1:=X2;X2:=G;
    End;

Begin
    Assign(F,Outputfile);
    Rewrite(F);
    Now:=Big;
    For I:=2 To N Do
        If Dis[1,I]+Len(1,I)<Now Then
            Begin
                Now:=Dis[1,I]+Len(1,I);
                X2:=I;
            End;
        X1:=1;
        Writeln(F,Now:0:2);
        P[1]:=1;P[2]:=1;
        Pr[1,1]:=X1;Pr[2,1]:=X2;
        D:=1;
        While (X1<>N) And (X2<>N) Do
            Begin
                Ok:=True;
                If X1+1<X2 Then
                    Begin
                        If Dis[X1+1,X2]+Len(X1,X1+1)=Dis[X1,X2] Then
                            Begin
                                Inc(X1);

```

```

        Inc(P[D]);
        Pr[D,P[D]]:=X1;
        Ok:=False;
    End;
End
Else
Begin
    I:=X2+1;
    While (I<=N) And (Dis[I,X2]+Len(X1,I)<>Dis[X1,X2]) Do
        Inc(I);
    If I<=N Then
    Begin
        Ok:=False;
        X1:=I;
        Inc(P[D]);
        Pr[D,P[D]]:=X1;
        Change;
    End;
    End;
    If Ok Then
    Begin
        I:=X2+1;
        While Dis[X1,I]+Len(I,X2)<>Dis[X1,X2] Do
            Inc(I);
        X2:=I;
        Inc(P[3-D]);
        Pr[3-D,P[3-D]]:=X2;
    End;
    End;
    While Pr[D,P[D]]<>N Do
    Begin
        Inc(P[D]);
        Pr[D,P[D]]:=Pr[D,P[D]-1]+1;
    End;

    For I:=1 To P[1] Do
        Write(F,Pr[1,I],' ');
    Writeln(F);
    For I:=1 To P[2] Do
        Write(F,Pr[2,I],' ');
    Close(F);
End;

Begin
    Init;
    Main;
    Print;
End.

```

```

Program Pro_3_2;
Const
    Inputfile  ='input.Txt';
    Outputfile ='Output.Txt';
    Max        =100;
    Big        =10000;

Var
    F          :Text;
    Po         :Array[1..Max,1..2] Of Integer;
    Qiu,Pr     :Array[1..Max] Of Byte;
    Lpr        :Integer;
    N          :Integer;
    Min        :Real;

Procedure Init;
Var
    I          :Integer;
Begin
    Assign(F,Inputfile);
    Reset(F);
    Readln(F,N);
    For I:=1 To N Do
        Readln(F,Po[I,1],Po[I,2]);
    Close(F);
End;

Function Len(P1,P2:Integer):Real;
Begin
    Len:=Sqrt(Sqr(Po[P1,1]-Po[P2,1])+Sqr(Po[P1,2]-Po[P2,2]));
End;

Procedure Search(Dep,Fr,Last:Byte;Al:Real);
Var
    I,P :Byte;
    Rs  :Real;

    Procedure Did;
    Var
        K :Byte;
    Begin
        K:=Fr+1;
        Rs:=0;
        P:=Last;
        While K<>I Do
            Begin
                Rs:=Rs+Len(P,K);
            End;
        End;
    End;
End;

```

```

        P:=K;
        Inc(K);
    End;
    Rs:=Rs+Len(Fr,I);
End;

Begin
    If Al>=Min Then Exit;
    If Fr=N Then
        Begin
            If Al+Len(Last,N)<Min Then
                Begin
                    Min:=Al+Len(Last,N);
                    Pr:=Qiu;
                    Lpr:=Dep-1;
                End;
            End
        Else
            For I:=Fr+1 To N Do
                Begin
                    Qiu[Dep]:=I;
                    Did;
                    Search(Dep+1,I,P,Al+Rs);
                End
            End;
        End;

Procedure Print;
Var
    I          :Byte;
    Se         :Set Of Byte;
Begin
    Assign(F,Outputfile);
    Rewrite(F);
    Writeln(F,Min:0:2);
    Se:=[1..N];
    For I:=1 To Lpr Do
        Begin
            Write(F,Pr[I], ' ');
            Se:=Se-[Pr[I]];
        End;
    Se:=Se+[1,N];
    Writeln(F);
    For I:=1 To N Do
        If I In Se Then
            Write(F,I, ' ');
    Close(F);
End;

```

```

Begin
  Init;
  Min:=Big;
  Search(1,1,1,0);
  Print;
End.

```

A.4 Ví dụ 4

Đề bài. Có một công ty vận chuyển bò đặt tại nông trại A , có một xe dùng để vận chuyển bò giữa bảy nông trại A, B, C, D, E, F, G . Khoảng cách giữa bảy nông trại được cho trong bảng sau:

	B	C	D	E	F	G
A	56	43	71	35	41	36
B	43	54	58	36	79	31
C	71	58	30	20	31	58
D	35	36	20	38	59	75
E	41	79	31	59	44	67
F	36	31	58	75	67	72

Mỗi buổi sáng, công ty phải quyết định thứ tự vận chuyển bò sao cho tổng quãng đường nhỏ nhất. Quy tắc:

1. Xe luôn xuất phát từ trụ sở công ty ở nông trại A , và sau khi mọi nhiệm vụ hoàn thành phải quay về trụ sở.
2. Xe mỗi lần chỉ chở một con bò.
3. Mỗi nhiệm vụ vận chuyển được cho bởi một cặp chữ cái, lần lượt biểu diễn bò sẽ được chở từ nông trại nào đến nông trại nào.

Nhiệm vụ của bạn là viết chương trình, với một tập nhiệm vụ cho trước, tìm lộ trình ngắn nhất để hoàn thành mọi nhiệm vụ, xuất phát từ nông trại A và cuối cùng trở về nông trại A .

Dữ liệu vào. Các nhiệm vụ đặt trong tệp `Delvr.Txt`. Dòng đầu là tổng số nhiệm vụ N , $1 \leq N \leq 12$. Từ dòng thứ hai, mỗi nhiệm vụ gồm một cặp chữ cái cách nhau bởi một dấu cách; chữ cái đầu biểu diễn nông trại nơi con bò đang ở, chữ cái thứ hai biểu diễn nông trại đích. Chương trình phải chạy không quá 60 giây.

Ví dụ vào.

```

5
FC
GB
BD
AE
GA

```

Dữ liệu ra. In ra màn hình độ dài lộ trình ngắn nhất, và hiển thị thứ tự các nông trại mà xe đi qua.

Ví dụ ra.

368

AEFCGBDGA

Ghi chú. Bài này lấy từ tạp chí *Olympic tin học*, số 1–2 năm 1998. Trong phần chính, bài này có ba thuật toán: quy hoạch động, tìm kiếm, và tìm kiếm cải tiến; lần lượt ứng với các chương trình Pro_4_1.Pas, Pro_4_2.Pas, Pro_4_3.Pas. Ngoài ra có năm dữ liệu thử; tệp vào và ra là Data_4_i.Txt, Sol_4_i.Txt, đặt trong thư mục Data.

Chương trình.

```
Program Pro_4_1;
Const
  Map:Array['A'..'G','A'..'G'] Of Byte=
    ((0,56,43,71,35,41,36),
     (56,0,54,58,36,79,31),
     (43,54,0,30,20,31,58),
     (71,58,30,0,38,59,75),
     (35,36,20,38,0,44,67),
     (41,79,31,59,44,0,72),
     (36,31,58,75,67,72,0));

  Inputfile  ='d1.Inp';
  Outputfile ='sol_4_1.Txt';

Type
  Kus      =Array[0..4095] Of Word;
  Dirs     =Array[0..4095] Of Byte;
  Ses      =Set Of 1..12;

Var
  F        :Text;
  N        :Integer;
  Wu       :Array[0..12,1..2] Of Char;
  Dis      :Array[0..12,0..12] Of Integer;
  Ku       :Array[1..12] Of ^kus;
  Dir      :Array[1..12] Of ^dirs;

Procedure Init;
Var
  I        :Integer;
  Ch       :Char;
Begin
  Assign(F,Inputfile);
  Reset(F);
  Readln(F,N);
  For I:=1 To N Do
```

```

    Readln(F,Wu[I,1],Ch,Wu[I,2]);
    Close(F);
End;

Procedure Prepare;
Var
    I,J          :Integer;
Begin
    Wu[0,1]:='A';Wu[0,2]:='A';
    For I:=0 To N Do
        For J:=0 To N Do
            Dis[I,J]:=Map[Wu[I,2],Wu[J,1]];
End;

Procedure Main;
Var
    Last,I,K,What :Word;
    S              :Ses;

Function Num(S:Ses):Word;
Var
    X      :Word Absolute S;
Begin
    Num:=X Div 2;
End;

Procedure Did(Dep,From:Byte;S:Ses);
Var
    I      :Byte;
Begin
    If Dep>K Then
        Begin
            For I:=1 To N Do
                If (I In S) And
                    (Ku[I]^[Num(S-[I])]+Dis[What,I]<Ku[What]^[Num(S)]) Then
                    Begin
                        Ku[What]^[Num(S)]:=Ku[I]^[Num(S-[I])]+Dis[What,I];
                        Dir[What]^[Num(S)]:=I;
                    End;
            End
        End
    Else
        For I:=From+1 To N Do
            If I<>What Then
                Did(Dep+1,I,S+[I]);
        End;

Begin
    For I:=1 To N Do

```

```

Begin
  New(Ku[I]);
  New(Dir[I]);
  Fillchar(Ku[I]^,Sizeof(Ku[I]^),$ff);
End;
For I:=1 To N Do
  Ku[I]^0:=Dis[I,0];

For K:=1 To N-1 Do
  For What:=1 To N Do
    Did(1,0,[]);

S:=1..N;
K:=60000;
For I:=1 To N Do
  If Dis[0,I]+Ku[I]^[Num(S-[I])]<K Then
    Begin
      K:=Dis[0,I]+Ku[I]^[Num(S-[I])];
      What:=I;
    End;
For I:=1 To N Do
  Inc(K,Map[Wu[I,1],Wu[I,2]]);

Assign(F,Outputfile);
Rewrite(F);
Writeln(F,K);
Write(F,'A ');
Last:=0;
For I:=1 To N Do
  Begin
    If Wu[Last,2]<>Wu[What,1] Then Write(F,Wu[What,1],' ');
    Write(F,Wu[What,2],' ');
    Exclude(S,What);
    Last:=What;
    What:=Dir[What]^[Num(S)];
  End;
If 'A'<>Wu[Last,2] Then Write(F,'A');
Close(F);
End;

Begin
  Init;
  Prepare;
  Main;
End.

Program Pro_4_2;

```

```

Const
  Map:Array['A'..'G','A'..'G'] Of Byte=
    ((0,56,43,71,35,41,36),
     (56,0,54,58,36,79,31),
     (43,54,0,30,20,31,58),
     (71,58,30,0,38,59,75),
     (35,36,20,38,0,44,67),
     (41,79,31,59,44,0,72),
     (36,31,58,75,67,72,0));
  Inputfile  ='d1.Inp';
  Outputfile ='data_4_1.Txt';

Var
  F          :Text;
  N          :Integer;
  Min        :Word;
  Wu         :Array[0..12,1..2] Of Char;
  Dis        :Array[0..12,0..12] Of Integer;
  Qiu,Pr     :Array[0..14] Of Byte;
  Se         :Set Of Byte;

Procedure Init;
Var
  I          :Integer;
  Ch         :Char;
Begin
  Assign(F,Inputfile);
  Reset(F);
  Readln(F,N);
  For I:=1 To N Do
    Readln(F,Wu[I,1],Ch,Wu[I,2]);
  Close(F);
End;

Procedure Prepare;
Var
  I,J        :Integer;
Begin
  Wu[0,1]:='A';Wu[0,2]:='A';
  For I:=0 To N Do
    For J:=0 To N Do
      Dis[I,J]:=Map[Wu[I,2],Wu[J,1]];
End;

Procedure Search(Dep:Byte;Al:Word);
Var
  I          :Byte;
Begin

```

```

If Al>=Min Then Exit;
If Dep=N+1 Then
Begin
  If Dis[Qiu[N],0]+Al<Min Then
  Begin
    Min:=Dis[Qiu[N],0]+Al;
    Pr:=Qiu;
  End;
End
Else
  For I:=1 To N Do
    If I In Se Then
      Begin
        Qiu[Dep]:=I;
        Exclude(Se,I);
        Search(Dep+1,Al+Dis[Qiu[Dep-1],I]);
        Include(Se,I);
      End;
    End;
  End;

Procedure Print;
Var
  I          :Byte;
Begin
  For I:=1 To N Do
    Inc(Min,Map[Wu[I,1],Wu[I,2]]);
  Assign(F,Outputfile);
  Rewrite(F);
  Writeln(F,Min);
  Write(F,'A ');
  For I:=1 To N Do
    Begin
      If Wu[Pr[I-1],2]<>Wu[Pr[I],1] Then Write(F,Wu[Pr[I],1],' ');
      Write(F,Wu[Pr[I],2],' ');
    End;
  If 'A'<>Wu[Pr[N],2] Then Write(F,'A');
  Close(F);
End;

Begin
  Init;
  Prepare;
  Se:=[1..12];
  Qiu[0]:=0;
  Min:=60000;
  Search(1,0);
  Print;
End.

```

Program Pro_4_3;

Const

```
Map:Array['A'..'G','A'..'G'] Of Byte=
  ((0,56,43,71,35,41,36),
   (56,0,54,58,36,79,31),
   (43,54,0,30,20,31,58),
   (71,58,30,0,38,59,75),
   (35,36,20,38,0,44,67),
   (41,79,31,59,44,0,72),
   (36,31,58,75,67,72,0));
Inputfile  ='d1.Inp';
Outputfile ='data_4_1.Txt';
```

Type

```
Kus      =Array[0..4095] Of Word;
Ses      =Set Of 1..12;
```

Var

```
F        :Text;
N        :Integer;
Min       :Word;
Big,I     :Word;
Wu       :Array[0..12,1..2] Of Char;
Dis      :Array[0..12,0..12] Of Integer;
Ku       :Array[1..12] Of ^kus;
Qiu      :Array[1..13] Of Byte;
Se       :Ses;
```

Procedure Init;

Var

```
I        :Integer;
Ch       :Char;
```

Begin

```
Assign(F,Inputfile);
Reset(F);
Readln(F,N);
For I:=1 To N Do
  Readln(F,Wu[I,1],Ch,Wu[I,2]);
Close(F);
```

End;

Procedure Prepare;

Var

```
I,J      :Integer;
```

Begin

```
Wu[0,1]:='A';Wu[0,2]:='A';
```

```

For I:=0 To N Do
  For J:=0 To N Do
    Dis[I,J]:=Map[Wu[I,2],Wu[J,1]];

For I:=1 To N Do
Begin
  New(Ku[I]);
  Fillchar(Ku[I]^,Sizeof(Ku[I]^),$ff);
End;
Big:=Ku[1]^[1];
End;

Function Num(S:Ses):Word;
Var
  X          :Word Absolute S;
Begin
  Num:=X Div 2;
End;

Procedure Search(Dep:Byte;Al:Word);
Var
  I          :Byte;
  D          :Word;
Begin
  If Al>=Min Then Exit;
  If Ku[Qiu[Dep-1]]^[Num(Se)]<Big Then
  Begin
    If Al+Ku[Qiu[Dep-1]]^[Num(Se)]<Min Then
      Min:=Al+Ku[Qiu[Dep-1]]^[Num(Se)];
    End
  Else
    If Dep>N Then
    Begin
      Ku[Qiu[Dep-1]]^[Num(Se)]:=Dis[Qiu[Dep-1],0];
      If Al+Ku[Qiu[Dep-1]]^[Num(Se)]<Min Then
        Min:=Al+Ku[Qiu[Dep-1]]^[Num(Se)];
      End
    Else
    Begin
      D:=Big;
      For I:=1 To N Do
        If I In Se Then
        Begin
          Exclude(Se,I);
          Qiu[Dep]:=I;
          Search(Dep+1,Al+Dis[Qiu[Dep-1],Qiu[Dep]]);
          If Ku[I]^[Num(Se)]+Dis[Qiu[Dep-1],Qiu[Dep]]<D Then
            D:=Ku[I]^[Num(Se)]+Dis[Qiu[Dep-1],Qiu[Dep]];

```

```

        Include(Se,I);
    End;
    Ku[Qiu[Dep-1]]^[Num(Se)]:=D;
End;

End;

Procedure Print;
Var
    I,J,Last    :Integer;
    D           :Word;
Begin
    Se:=[1..N];
    J:=1;
    While Dis[0,J]+Ku[J]^[Num(Se-[J])]<>Min Do
        Inc(J);
    D:=Min;
    For I:=1 To N Do
        Inc(D,Map[Wu[I,1],Wu[I,2]]);
    Assign(F,Outputfile);
    Rewrite(F);
    Writeln(F,D);
    Write(F,'A ');
    Dec(Min,Dis[0,J]);
    Last:=0;

    For I:=1 To N Do
    Begin
        If Wu[J,1]<>Wu[Last,2] Then
            Write(F,Wu[J,1],' ');
        Write(F,Wu[J,2],' ');
        If I<>N Then
            Begin
                Last:=J;
                Exclude(Se,J);
                J:=1;
                While (Not (J In Se)) Or
                    (Ku[J]^[Num(Se-[J])]+Dis[Last,J]<>Min) Do
                    Inc(J);
                Dec(Min,Dis[Last,J]);
            End;
        End;
        If Wu[J,2]<>'A' Then Write(F,'A');
    Close(F);
End;

Begin
    Init;
    Prepare;

```

```

Se:=[1..N];
Min:=Big;
For I:=1 To N Do
Begin
  Se:=[1..N]-[I];
  Qiu[1]:=I;
  Search(2,Dis[0,I]);
End;
Print;
End.

```

A.5 Ví dụ 5

Đề bài. Một số cấu trúc phức tạp của sinh vật có thể biểu diễn bằng dãy các nguyên tố, còn một nguyên tố được biểu diễn bằng một chuỗi chữ cái in hoa. Một vấn đề của các nhà sinh học là phân giải một dãy dài như vậy thành một dãy các nguyên tố. Với tập nguyên tố P cho trước, nếu có thể chọn N nguyên tố $P_1, P_2, P_3, \dots, P_n$ từ đó, nối các chuỗi tương ứng theo thứ tự để được một chuỗi S , thì gọi S có thể được tạo bởi tập nguyên tố P . Khi chọn nguyên tố từ P , một nguyên tố có thể dùng nhiều lần hoặc không dùng. Ví dụ, dãy aBaBaCaBaaB có thể tạo bởi tập nguyên tố $\{a, aB, Ba, Ca, BBC\}$.

K ký tự đầu của một chuỗi được gọi là tiền tố của chuỗi đó, với độ dài K . Hãy viết chương trình: với tập nguyên tố P và chuỗi T nhập vào, tìm một tiền tố của T có thể tạo bởi tập nguyên tố P , sao cho độ dài tiền tố lớn nhất có thể, và xuất độ dài ấy.

Dữ liệu vào. Có hai tệp vào INPUT.TXT và DATA.TXT. Dòng đầu của INPUT.TXT là số nguyên tố N trong tập P , $1 \leq N \leq 100$. Sau đó có $2N$ dòng, mỗi hai dòng mô tả một nguyên tố: dòng đầu là độ dài L , $1 \leq L \leq 20$; dòng sau là chuỗi chữ cái in hoa độ dài L , biểu diễn nguyên tố ấy. Mọi nguyên tố đôi một khác nhau.

Tệp DATA.TXT mô tả chuỗi T . Mỗi dòng bắt đầu bằng một chữ cái in hoa; dòng cuối là ký tự ., biểu thị kết thúc chuỗi. Độ dài của T ít nhất là 1 và nhiều nhất không vượt quá 500000.

Dữ liệu ra. Tệp ra tên OUTPUT.TXT. Chỉ có một dòng là một số, biểu thị độ dài tiền tố dài nhất của T có thể tạo bởi P .

Ghi chú. Bài này là đề thi Olympic Tin học Quốc tế lần thứ tám. Chương trình ở dưới. Dữ liệu vào và ra mẫu là Data_5.Txt, Input_5.Txt, Sol_5.Txt trong thư mục Data.

Chương trình.

```

Program Pro_5;

Const
  Datafile   ='Data.Txt';
  Inputfile  ='Input.Txt';
  Outputfile ='Output.Txt';

Type
  Jis = Record

```

```

        Lst : Byte;
        St  : String[20];
    End;

Var
    F          :Text;
    Ji         :Array[1..100] Of Jis;
    Ch         :Array[0..21] Of Char;
    Bo         :Array[0..21] Of Boolean;
    N          :Integer;
    Tot        :Longint;
    Big        :Longint;

Procedure Init;
Var
    I          :Integer;
Begin
    Assign(F,Inputfile);
    Reset(F);
    Readln(F,N);
    For I:=1 To N Do
    Begin
        Readln(F,Ji[I].Lst);
        Readln(F,Ji[I].St);
    End;
    Close(F);
End;

Procedure Main;
Var
    I,J        :Integer;

    Procedure See;
    Var
        I      :Byte;
        X,Y    :Integer;
    Begin
        Bo[J]:=False;
        For I:=1 To N Do
        Begin
            X:=J;Y:=Ji[I].Lst;
            While (Y>0) And (Ji[I].St[Y]=Ch[X]) Do
            Begin
                Dec(X);
                Dec(Y);
                If X<0 Then X:=21;
            End;
            If (Y=0) And (Bo[X]) Then

```

```

        Begin
            Bo[J]:=True;
            Exit;
        End;
    End;
End;

Begin
    Assign(F,Datafile);
    Reset(F);
    Big:=0;
    Tot:=0;
    Bo[0]:=True;
    J:=0;
    While (Tot-Big<20) And (Ch[J]<>'.') Do
        Begin
            Inc(Tot);Inc(J);
            If J>21 Then J:=0;
            Readln(F,Ch[J]);
            If Ch[J]<>'.' Then
                Begin
                    See;
                    If Bo[J] Then Big:=Tot;
                End;
            End;
        End;
    Close(F);
End;

Procedure Print;
Begin
    Assign(F,Outputfile);
    Rewrite(F);
    Writeln(F,Big);
    Close(F);
End;

Begin
    Init;
    Main;
    Print;
End.

```

A.6 Ví dụ 6

Đề bài. Một công ty nhập vào một lô thùng, tổng số là N , $1 \leq N \leq 1000$, được đưa vào lần lượt bằng băng chuyền, rồi được xếp trong kho thành nhiều nhất P cột, $1 \leq P \leq 4$. Quá trình xếp này do một máy điều phối thực hiện, như Hình 8.

Biết các thùng nhập vào nhiều nhất thuộc M loại, $1 \leq M \leq 20$. Ông chủ tự nhiên muốn các thùng

được xếp càng đẹp càng tốt, tức là xếp các thùng cùng loại gần nhau nhất có thể. Theo sở thích của ông chủ, độ đẹp T được định nghĩa là

$$T = \sum (\text{số loại khác nhau nhìn thấy lần lượt trong mỗi cột}).$$

“Số loại khác nhau nhìn thấy lần lượt” nghĩa là nếu thùng thứ K khác loại với thùng thứ $K - 1$, thì tính thêm một loại. Ví dụ, cột 1 2 3 1 có số loại khác nhau nhìn thấy lần lượt là 4; cột 1 2 2 3 3 1 2 có số loại này là 5. Vì vậy T càng nhỏ thì càng đẹp, và ông chủ càng hài lòng.

Đã biết không hạn chế số thùng trong mỗi cột; có thể xếp mọi thùng vào một cột, cũng có thể xếp thành P cột. Nhưng do thùng được vận chuyển bằng băng chuyền nên thứ tự thùng là cố định: nếu thùng A đến trước thùng B , thì nếu A, B được xếp cùng một cột, A phải nằm trước B .

Hãy viết một chương trình cho máy điều phối, để máy dựa vào thông tin thùng do băng chuyền gửi tới mà điều phối, sao cho sau khi xếp thành P cột thì độ đẹp nhỏ nhất.

Dữ liệu vào. Tập vào tên INPUT.TXT. Nội dung gồm hai dòng. Dòng đầu có ba số theo thứ tự là N, M, P . Dòng thứ hai có N số, là loại của các thùng do băng chuyền đưa tới; số thứ K là loại của thùng thứ K .

Dữ liệu ra. Tập ra tên OUTPUT.TXT. Nội dung có hai dòng. Dòng đầu là độ đẹp T . Dòng thứ hai có N số, là một phương án điều phối; số thứ K biểu thị thùng thứ K sẽ đi đến cột nào.

Ghi chú. Bài này do tác giả tự đặt. Chương trình ở dưới; dữ liệu vào và ra mẫu là Data_6.Txt, Sol_6.Txt trong thư mục Data.

Chương trình.

```
Program Pro_6;
Const
  Fin = 'Input.txt';
  Fou = 'Output.txt';

Type
  Kus = Array[0..1160] Of Byte;
  Zis = Array[0..1160] Of Integer;

Var
  Ku      : Array[1..1000] Of ^kus;
  Shu,Shu1 : Array[1..1000] Of Byte;
  Zi      : Array[0..1] Of Zis;
  Use1    : Array[0..20,0..20,0..20] Of Integer;
  Use2    : Array[0..1160,1..3] Of Byte;
  Total,I,M,N,P,N1 : Integer;
  F       : Text;

Procedure Init;
Var
  I,J      :Integer;
Begin
  Assign(F,Fin);
```

```

Reset(F);
Readln(F,N1,M,P);
For I:=1 To N1 Do
    Read(F,Shu1[I]);
Shu[1]:=Shu1[1];
N:=1;
For I:=2 To N1 Do
    If Shu1[I]<>Shu1[I-1] Then
        Begin
            Inc(N);
            Shu[N]:=Shu1[I];
        End;
Close(F);
End;

Procedure Diduse;
Var
    Qs      :Array[1..3] Of Byte;
    Stop    :Byte;
    I,J,K    :Byte;

Procedure Serch(Depth,From:Byte);
Var
    I      :Byte;
Begin
    If Depth=Stop+1 Then
        Begin
            Inc(Total);
            Use2[Total,1]:=Qs[1];Use2[Total,2]:=Qs[2];
            Use2[Total,3]:=Qs[3];
            Use1[Qs[1],Qs[2],Qs[3]]:=Total;
            Use1[Qs[1],Qs[3],Qs[2]]:=Total;
            Use1[Qs[2],Qs[1],Qs[3]]:=Total;
            Use1[Qs[2],Qs[3],Qs[1]]:=Total;
            Use1[Qs[3],Qs[1],Qs[2]]:=Total;
            Use1[Qs[3],Qs[2],Qs[1]]:=Total;
        End
    Else
        For I:=From To M-1 Do
            Begin
                Qs[Depth]:=I;
                Serch(Depth+1,I+1);
            End;
End;

Begin
    Total:=0;
    Fillchar(Qs,Sizeof(Qs),0);

```

```

For I:=0 To M Do
  For J:=0 To M Do
    For K:=0 To M Do
      Use1[I,J,K]:=-1;
    Use2[0,1]:=0;Use2[0,2]:=0;Use2[0,3]:=0;
    Use1[0,0,0]:=0;
  For Stop:=1 To P-1 Do
    Serch(1,1);
End;

Procedure Main;
Var
  I,J,K,W1,W2,W3 :Integer;
  Q                :Array[1..4] Of Byte;
Begin
  For I:=1 To N Do
    Begin
      New(Ku[I]);
    End;
    Fillchar(Zi,Sizeof(Zi),0);
    Fillchar(Ku[1]^,Sizeof(Ku[1]^),0);
    Zi[0][0]:=1;
    Fillchar(Q,Sizeof(Q),0);
    For I:=2 To N Do
      Begin
        For J:=0 To Total Do
          If Zi[0][J]<>0 Then
            Begin
              W1:=0;
              For K:=1 To 3 Do
                Begin
                  Q[K]:=Use2[J,K];
                  If Q[K]>=Shu[I-1] Then Inc(Q[K]);
                  If Q[K]=Shu[I] Then W1:=K;
                End;
              Q[4]:=Shu[I-1];
              If W1<>0 Then
                Begin
                  W2:=Q[4];Q[4]:=Q[W1];Q[W1]:=W2;
                  For K:=1 To 3 Do
                    If Q[K]>Shu[I] Then Dec(Q[K]);
                  W3:=Use1[Q[1],Q[2],Q[3]];
                  If (Zi[1][W3]=0) Or (Zi[1][W3]>Zi[0][J]) Then
                    Begin
                      Zi[1][W3]:=Zi[0][J];
                      Ku[I]^W3:=Shu[I];
                    End;
                End;
            End;
          End;
        End
      End
    End
  End

```

```

Else
Begin
  For K:=1 To 4 Do
    If Q[K]>Shu[I] Then Dec(Q[K]);
  For K:=1 To 4 Do
    Begin
      Case K Of
        1:W1:=Use1[Q[2],Q[3],Q[4]];
        2:W1:=Use1[Q[1],Q[3],Q[4]];
        3:W1:=Use1[Q[1],Q[2],Q[4]];
        4:W1:=Use1[Q[1],Q[2],Q[3]];
      End;
      If W1<>-1 Then
        If (Zi[1][W1]=0) Or (Zi[1][W1]>Zi[0][J]+1) Then
          Begin
            Zi[1][W1]:=Zi[0][J]+1;
            W2:=Q[K];
            If W2>=Shu[I] Then Inc(W2);
            Ku[I]^W1:=W2;
          End;
        End;
      End;
    End;
  Zi[0]:=Zi[1];
  Fillchar(Zi[1],Sizeof(Zi[1]),0);
End;
End;

Procedure Print;
Var
  Pr,Pr1      :Array[1..1000] Of Integer;
  Now,Q        :Array[1..4] Of Byte;
  I,J,K,Ps,W1,W2 :Integer;
Begin
  Ps:=1001;
  For J:=0 To Total Do
    If (Zi[0][J]<>0) And (Zi[0][J]<Ps) Then
      Begin
        Ps:=Zi[0][J];
        K:=J;
      End;
  Assign(F,Fou);
  Rewrite(F);
  Writeln(F,Ps);
  Now[2]:=Use2[K,1];
  Now[3]:=Use2[K,2];
  Now[4]:=Use2[K,3];
  For I:=2 To 4 Do

```

```

    If Now[I]>=Shu[N] Then Inc(Now[I]);
Now[1]:=Shu[N];
Pr[N]:=1;
For K:=N-1 Downto 1 Do
Begin
    J:=1;
    While Now[J]<>Shu[K+1] Do
        Inc(J);
    W2:=0;
    For I:=1 To 4 Do
        If I<>J Then
            Begin
                Inc(W2);
                Q[W2]:=Now[I];
                If Q[W2]>=Shu[K+1] Then Dec(Q[W2]);
            End;
    W1:=Use1[Q[1],Q[2],Q[3]];
    Now[J]:=Ku[K+1]^W1;
    For I:=1 To 4 Do
        If Now[I]=Shu[K] Then
            Pr[K]:=I;
    End;
    J:=1;
    Pr1[1]:=Pr[1];
    For I:=2 To N1 Do
    Begin
        If Shu1[I]<>Shu1[I-1] Then Inc(J);
        Pr1[I]:=Pr[J];
    End;
    For I:=1 To N1 Do
        Write(F,Pr1[I], ' ');
    Close(F);
End;

Begin
    Init;
    Diduse;
    Main;
    Print;
End.

```

Các chương trình trên đều được kiểm thử thành công trong môi trường Pascal 7.0.

References

- [1] *Hướng dẫn thi Olympic tin học quốc tế và toàn quốc dành cho thanh thiếu niên: thuật toán và thiết kế chương trình trong tổ hợp*, Wu Wenhua và Wang Jiande biên soạn, Nhà xuất bản Đại học Thanh Hoa.
- [2] *Tạp chí Olympic tin học*, số 1–2 năm 1998.

- [3] *Đại toàn tập cấu trúc dữ liệu máy tính và thuật toán thực dụng*, Mou Ren chủ biên, Công ty máy tính Hy vọng Bắc Kinh.
- [4] *Phân tích đề thi Olympic tin học trong nước và quốc tế năm 1996*, Wu Wenhui và Wang Jiande biên soạn, Nhà xuất bản Đại học Bắc Kinh.